2026年4月20日

OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

OshiruZine-AI

2026年4月20日 号

目次

1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. GPT-5・Gemini 2.5・Claude 4——2026年春のフロンティアモデル徹底比較と選定ガイド　*特集（約4000字）*
3. AIエージェント時代のセキュリティリスク——プロンプトインジェクションからツール連鎖攻撃まで　*スタンダード（約2000字）*
4. EU AI Act本格施行から半年——日本企業が今やるべきコンプライアンス対応チェックリスト　*ショート（500〜1000字）*

Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

2026年に入り、AIコーディングエージェントは「補助ツール」から「開発チームの一員」へと明確にフェーズが変わった。特にAnthropic の **Claude Code** と OpenAI の **Codex** は、ターミナルネイティブなエージェント型開発体験を提供する双璧として、多くの現場で本格運用が進んでいる。本コラムでは、両ツールを日常的に使い込んできた筆者の経験と、2026年前半時点での最新動向を踏まえ、実務で差がつくベストプラクティスを整理する。

エージェント型開発が「当たり前」になった背景

2025年後半、Claude Code は Claude 4 系モデルとの統合により、リポジトリ全体のコンテキストを長時間保持したまま複数ファイルをまたぐリファクタリングを一気通貫で行えるようになった。一方の Codex（旧 Codex CLI）も、2026年初頭の大型アップデートで `codex --full-auto` モードを正式リリースし、サンドボックス内でのビルド・テスト・修正のループを自律的に回せるようになった。

両者に共通するのは、「**人間が指示を出し、エージェントが実行し、結果を人間がレビューする**」というワークフローが標準化された点だ。GitHub の調査によれば、2026年Q1時点でフォーチュン500企業の約62%がいずれかのAIコーディングエージェントをCI/CDパイプラインに組み込んでいるという。もはや「使うかどうか」ではなく「どう使いこなすか」の勝負になっている。

CLAUDE.md と codex.md —— プロジェクト規約ファイルの設計が鍵

Claude Code を使う上で最も効果が大きいプラクティスは、`CLAUDE.md`（**旧 INSTRUCTIONS.md**）の丁寧な設計だ。ここにはコーディング規約、アーキテクチャ方針、禁止パターン、テスト戦略などを記述する。2026年2月のアップデートで階層的な読み込み（リポジトリルート → サブディレクトリ → ユーザーホーム）が正式サポートされ、モノレポでもパッケージごとに異なる規約を適用できるようになった。

Codex 側でも同様に `codex.md`（あるいは `.codex/instructions.md`）が導入されている。ポイントは以下の3点だ。

1. **具体的な「やるな」を書く** —— 「きれいなコードを書け」ではなく「`any` 型を使うな」「直接 DOM 操作するな」のように否定形で明示する方がエージェントの逸脱を防げる。
2. **テストコマンドを明記する** —— `npm test`、`pytest -x` など、エージェントが自律的に検証ループを回すために必要な実行コマンドを正確に書く。
3. **アーキテクチャ図をテキストで埋め込む** —— Mermaid 記法やASCIIアートでレイヤー構成を示すと、エージェントがファイル配置の判断を誤りにくくなる。

プロンプトエンジニアリングから「タスク分割エンジニアリング」へ

2025年までは「いかに良いプロンプトを書くか」が議論の中心だった。しかし2026年現在、エージェントの能力向上に伴い、重要度が増しているのは **タスクの粒度設計** だ。

Claude Code の `/task` コマンドや Codex の `--plan` フラグを使えば、エージェントにまず実行計画を生成させ、人間が承認してから実行に移る二段階フローを取れる。筆者の経験則では、**1タスクあたりの変更が300行以内** に収まるよう分

割するのが最もレビュー効率が良い。大きすぎるとエージェントが中盤でコンテキストを見失い、小さすぎるとオーバーヘッドが増える。

また、Claude Code の最新版では **並列サブエージェント実行**（`--multi-agent`）が安定し、フロントエンドとバックエンドの変更を別エージェントに同時に投げて最後にマージする運用が現実的になった。この場合、タスク間のインターフェース定義（APIスキーマやTypeScriptの型定義）を事前に固めておくことが成功の前提条件となる。

自律実行時代のセキュリティとガードレール

エージェントに広い権限を与えるほど生産性は上がるが、リスクも比例して増大する。2026年3月には、あるOSSプロジェクトでAIエージェントが依存ライブラリの悪意あるバージョンを自動的に `npm install` してしまったインシデントが報告され、業界に衝撃を与えた。

対策として確立されつつあるベストプラクティスは次の通りだ。

- **ネットワーク隔離されたサンドボックスでの実行を必須化する** —— Codex はデフォルトでネットワーク無効のサンドボックスを提供している。Claude Code でも Docker ベースの隔離環境が推奨構成となった。
- **許可リスト方式のファイルアクセス** —— `.claude/settings.json` や `.codex/permissions.yml` で、エージェントが読み書きできるディレクトリを明示的にホワイトリスト化する。
- **差分レビューの自動ゲート** —— CI上でエージェントが生成したPRに対して、静的解析・依存関係チェック・SAST（静的アプリケーションセキュリティテスト）を通過しなければマージできないフローを組む。

両ツールの使い分け —— 2026年春時点の所感

筆者の現時点での使い分けはこうだ。**長いコンテキストが必要な大規模リファクタリングやコードベース全体の理解が求められるタスクには Claude Code、明**

確に定義された単機能の実装やバグ修正で素早く回したい場合は Codex がフィットする。Claude 4 系の圧倒的なコンテキスト長（実効100万トークン超）は複雑な依存関係の把握に強く、Codex の高速なサンドボックス実行ループは短サイクルのイテレーションに向いている。

もちろん、両ツールとも急速に進化しており、この優劣は半年後には変わっている可能性が高い。重要なのは **特定のツールに依存しすぎず、プロジェクト規約ファイルとCI/CDのガードレールをツール非依存に設計しておくこと** だ。そうすれば、エージェントを切り替えても開発プロセスの骨格は揺るがない。

まとめ —— 人間の役割は「設計」と「判断」に集約される

AIコーディングエージェントの成熟により、エンジニアの仕事は「コードを書く」から「アーキテクチャを設計し、エージェントの出力を判断する」方向へ確実にシフトしている。CLAUDE.md の設計力、タスク分割の巧拙、セキュリティガードレールの構築 —— これらが2026年の開発生産性を左右する本質的なスキルだ。ツールは進化し続ける。だからこそ、その進化を正しく御するための「メタスキル」を磨き続けたい。

GPT-5・Gemini 2.5・Claude 4——2026年春のフロンティアモデル徹底比較と選定ガイド

「どのモデルを使えばいいのか、もう分からない」——2026年春、エンジニア同士の会話で最も頻繁に飛び交うのがこの一言だろう。OpenAIのGPT-5、GoogleのGemini 2.5 Ultra、AnthropicのClaude 4。三大プレイヤーがわずか数か月の間に相次いでフロンティアモデルを投入し、ベンチマークの数字は日々塗り替え

られている。選択肢が増えること自体は歓迎すべきだが、プロダクションで採用するモデルを1つに絞らなければならない現場のエンジニアにとって、この"選択肢の爆発"は頭痛のタネでしかない。

本稿では、2026年前半時点で入手可能な3つのフロンティアモデルを、性能・コスト・ライセンス・運用面から横断的に比較し、ユースケース別の選定指針を提示する。読者が明日のプルリクエストで迷わないための、実践的なガイドとしてお役立ていただければ幸いだ。

2026年春、フロンティアモデル競争の現在地

2025年後半から2026年初頭にかけて、大規模言語モデル（LLM）の競争環境は一変した。

OpenAIは2025年末にGPT-5をリリースし、推論性能・マルチモーダル対応の両面でGPT-4oから大幅なジャンプを見せた。続いてGoogleが2026年2月にGemini 2.5 Ultraを一般提供開始。100万トークンを超えるコンテキストウィンドウと、ネイティブなマルチモーダル統合が話題をさらった。そしてAnthropicが2026年3月にClaude 4（Opus / Sonnet）をローンチし、安全性と長文推論の精度でベンチマーク上位を奪取した。

三者はいずれも「推論能力の深化」「マルチモーダルのネイティブ統合」「エージェント的な自律タスク実行」という3つの方向に進化しており、表面上のスペックシートだけでは差が見えにくくなっている。だからこそ、実務レベルのきめ細かい比較が不可欠だ。

スペック比較：GPT-5 vs Gemini 2.5 Ultra vs Claude 4

まずは基本スペックを整理しよう。

項目	GPT-5	Gemini 2.5 Ultra	Claude 4 Opus
コンテキスト長	256Kトークン	2Mトークン	512Kトークン
マルチモーダル	テキスト/画像/音声/動画	テキスト/画像/音声/動画/コード実行	テキスト/画像/音声/PDF
推論モード	標準 / Deep Thinking	標準 / Deep Research	標準 / 拡張思考
ツール使用	Function Calling / Code Interpreter / Web Browse	統合ツール群（Google検索・コード実行・Google Workspace連携）	Tool Use / Computer Use / MCP対応
API入力単価（100万トークン）	約$10	約$7	約$12
API出力単価（100万トークン）	約$30	約$21	約$36
レイテンシ（TTFT目安）	0.8〜1.5秒	0.6〜1.2秒	1.0〜2.0秒
ファインチューニング	対応（GPT-5 mini）	対応（Gemini 2.5 Flash）	2026年Q2対応予定

※価格・レイテンシは2026年4月時点のAPI公式ドキュメントおよびコミュニティベンチマークに基づく概算値。実際のワークロードにより変動する。

ベンチマーク性能：何が得意で、何が苦手か

コーディング能力

SWE-bench Verified、HumanEval+、LiveCodeBenchなどのコーディングベンチマークでは、GPT-5とClaude 4 Opusが僅差でトップを争っている。GPT-5はコード生成の「初手の正確さ」に強みがあり、ゼロショットで動くコードを出す確率が高い。一方、Claude 4 Opusは複雑なリファクタリングや大規模コードベースの理解に秀でており、既存プロジェクトへの統合タスクでは一歩リードする印象だ。Gemini 2.5 Ultraはコード実行環境をネイティブに組み込んでいるため、「書いて・実行して・修正する」のサイクルをモデル内で完結できる点がユニークだが、純粋なコード生成品質では他の2つにやや劣る場面もある。

長文理解・要約

ここはGemini 2.5 Ultraの独壇場だ。200万トークンのコンテキストウィンドウは伊達ではなく、数百ページの技術文書や数時間分の議事録音声を丸ごと投入して要約・質問応答ができる。Claude 4 Opusも512Kトークンのウィンドウと「精読型」の注意力で高品質な長文要約を返すが、Geminiの物量にはかなわない。GPT-5の256Kトークンは十分に広いものの、超長文タスクでは情報の取りこぼしが散見される。

数学・科学推論

Claude 4 Opusの拡張思考モードが頭一つ抜けている。MATH、GPQA Diamond、FrontierMathといった難関ベンチマークで最高スコアを記録しており、ステップバイステップの推論過程の透明性も高い。GPT-5のDeep Thinkingモードも優秀だが、推論ステップが長くなるとハルシネーションの頻度がやや上がる傾向がある。

マルチモーダル

動画理解と音声処理ではGemini 2.5 Ultraが最も成熟している。YouTubeエコシステムとの統合で鍛えられたモデルの面目躍如だ。画像理解の精度ではGPT-5

が安定しており、図表の読み取りやOCR的なタスクでの信頼性が高い。Claude 4はPDFのネイティブ理解に強く、ビジネス文書の処理ではベストチョイスになることが多い。

コストと運用：本番環境で効いてくる差

ベンチマークの数字だけでモデルを選ぶエンジニアはいないだろう。本番運用で効いてくるのは、コスト構造、レート制限、SLA、そしてデータプライバシーだ。

コスト効率では、Gemini 2.5 Ultraが最も攻めた価格設定をしている。Google Cloudのコミットメント契約と組み合わせると、さらに20〜30%のディスカウントが適用されるケースもある。OpenAIもGPT-5 miniという軽量バリアントを用意しており、品質を若干トレードオフにすれば入力$1.5/100万トークン程度まで下げられる。Claude 4はOpusの単価が最も高いが、Sonnetバリアントを選べば入力$3/100万トークン前後に抑えられ、多くのタスクでOpusに迫る品質を発揮する。

レート制限とスループットは、大量リクエストを捌くバッチ処理で重要になる。OpenAIのTier 5以上では1分あたり数十万トークンのスループットが確保でき、Geminiもvertex AI経由で同等の枠を提供している。Anthropicは2026年初頭にバッチAPIを強化し、50%のコスト削減と引き換えに24時間以内の非同期処理を提供するオプションを追加した。

データプライバシーとコンプライアンスの面では、3社ともAPI経由のデータをモデル訓練に使用しないポリシーを明確にしているが、規制業種（金融・医療・政府）での採用実績はAnthropicとMicrosoftのAzure OpenAI Serviceがリードしている。Googleも主権クラウド（Sovereign Cloud）対応を進めているが、欧州・日本のエンタープライズではまだAzure OpenAIの採用が目立つ。

ライセンスとエコシステム

モデルそのもののライセンスに加え、周辺エコシステムの充実度も選定に影響する。

- **OpenAI**：ChatGPT / API / Azure OpenAI Serviceという3チャネルの提供。企業向けにはAzure経由が事実上の標準で、既存のMicrosoftスタックとの統合がスムーズ。Assistants API v3でエージェント構築のフレームワークも成熟してきた。
- **Google**：Vertex AI / AI Studio / Google Workspaceへの組み込み。BigQueryやCloud Runとのシームレスな連携が魅力で、データパイプラインの中にLLMを組み込むアーキテクチャが設計しやすい。Androidデバイスへのオンデバイス展開（Gemini Nano 2.5）も差別化ポイント。
- **Anthropic**：API直接提供 / Amazon Bedrock / Google Cloud Vertex AI（パートナー提供）。クラウドプロバイダーを選べる柔軟性がある。MCP（Model Context Protocol）の普及により、外部ツール連携のオープンスタンダード化を牽引している点も見逃せない。

ユースケース別・選定ガイドライン

ここからが本稿の核心だ。代表的なユースケースごとに、筆者の推奨モデルとその理由を示す。

1. コード生成・開発支援ツール

推奨：Claude 4 Sonnet → GPT-5 mini

IDE統合やコードレビュー自動化には、コスト効率と品質のバランスが重要。Claude 4 Sonnetはコード理解の深さとコストのバランスが最も良い。GPT-5 miniも有力な代替で、特に既存のGitHub Copilot（GPT-5ベース）とのエコシステム統合を考慮するなら第一候補になる。

2. 大規模ドキュメント処理・RAG

推奨：Gemini 2.5 Ultra

200万トークンのコンテキストウィンドウにより、チャンキング戦略の設計負荷を劇的に軽減できる。特にRAGパイプラインでの「検索精度のボトルネック」を、ロングコンテキストで力業で突破できるのはGemini固有の強みだ。ただし、コストが気になる場合はGemini 2.5 Flashで十分なケースも多い。

3. 科学論文の解析・研究支援

推奨：Claude 4 Opus（拡張思考モード）

数式の解釈、論理的整合性のチェック、複雑な推論チェーンの構築で最も信頼性が高い。拡張思考モードでは推論過程が可視化されるため、研究者がモデルの判断を検証しやすい。

4. マルチモーダルアプリケーション（動画・音声解析）

推奨：Gemini 2.5 Ultra

動画のフレームレベル理解、音声の自動文字起こし＋意味解析が最も高品質。製造業の外観検査、教育コンテンツの自動字幕生成、セキュリティ映像の異常検知といった領域で強い。

5. カスタマーサポート・チャットボット

推奨：GPT-5 mini → Gemini 2.5 Flash

大量のリクエストを低コスト・低レイテンシで捌く必要があるため、軽量モデルの出番。GPT-5 miniはレスポンスの自然さ、Gemini 2.5 Flashはコスト効率で優位。安全性を最重視するならClaude 4 Sonnetも検討に値する。

6. エージェント型アプリケーション

推奨：Claude 4 Opus + MCP → GPT-5 + Assistants API

2026年のトレンドである「AIエージェント」を構築する場合、ツール呼び出しの正確性と安全性が鍵になる。Claude 4のComputer UseとMCP対応はエージェント設計において最もオープンかつ柔軟。一方、Microsoft/OpenAIのAssistants API v3は統合的なエージェントフレームワークとして完成度が高く、Azure上でのエンタープライズ運用には適している。

実務上の選定フローチャート

最後に、実務で素早く判断するためのフローを整理しておく。

1. **コンテキスト長が50万トークンを超える要件があるか？** → Yes → Gemini 2.5 Ultra
2. **既存インフラがAzure / Microsoft中心か？** → Yes → GPT-5（Azure OpenAI Service経由）
3. **安全性・コンプライアンス要件が厳格か？** → Yes → Claude 4（Constitutional AIによる安全性設計）
4. **コスト最適化が最優先か？** → Yes → Gemini 2.5 Flash or GPT-5 mini
5. **上記いずれにも該当しない汎用ケース** → Claude 4 Sonnet（品質・コスト・安全性のバランスが最も良い）

もちろん、これは出発点に過ぎない。プロダクション導入前には必ず自社データでの評価（evals）を実施し、ベンチマークではなく「自分たちのタスクでの品質」を基準に判断すべきだ。

おわりに：「最強モデル」は存在しない

2026年春の時点で断言できるのは、万能の最強モデルは存在しないということだ。GPT-5は総合力とエコシステムの厚み、Gemini 2.5 Ultraはマルチモーダル

とロングコンテキスト、Claude 4は推論の深さと安全性——それぞれに明確な強みがある。

そして、この勢力図は半年後には確実に変わっている。モデル競争のペースを考えれば、特定モデルにロックインしないアーキテクチャ設計——抽象化レイヤーの導入、モデル切り替えを前提としたプロンプト管理、複数プロバイダーへのフェイルオーバー——がこれまで以上に重要になる。

フロンティアモデルの進化は止まらない。しかし、どのモデルを「選ぶ」かよりも、どう「使いこなす」かが、エンジニアの腕の見せどころであることは、いつの時代も変わらない。

AIエージェント時代のセキュリティリスク——プロンプトインジェクションからツール連鎖攻撃まで

あなたのチームが運用しているAIエージェントは、今この瞬間も外部APIを叩き、データベースを参照し、コードを生成して実行しているかもしれない。2026年に入り、自律型AIエージェントの本番導入は「実験」から「常態」へと完全にフェーズが移った。OpenAIの Operator、Googleの Project Mariner、Anthropicの Claude Agent SDK——主要プレイヤーが相次いでエージェントフレームワークを正式リリースし、企業のバックオフィスから開発パイプラインまで、ツール連鎖型のAIワークフローが急速に浸透している。しかし、その裏側では従来のWebアプリケーションセキュリティとはまったく異質の脅威が顕在化しつつある。本稿では、2026年現在までに報告された実例と最新の防御パターンを体系的に整理する。

プロンプトインジェクション――依然として「未解決の根本問題」

プロンプトインジェクションは2023年頃から指摘されてきた古典的な攻撃だが、エージェント時代に入って破壊力が桁違いに増した。単体のチャットボットであれば、不正な応答を引き出す程度の被害で済む場合が多い。しかしエージェントは「思考→ツール呼び出し→結果の解釈→次のアクション」というループを自律的に回す。つまり、一度インジェクションが成功すると、その指示がツール実行という実世界のアクションに直結する。

2025年末にOWASPが更新した「LLM Top 10 v2.0」でも、プロンプトインジェクションは引き続き第1位にランクされた。特に問題視されているのが**間接プロンプトインジェクション**だ。エージェントが取得するWebページ、メール本文、PDFドキュメントなどの外部データに攻撃命令を仕込み、エージェントのコンテキストウィンドウに侵入させる手法である。2026年2月には、ある企業の社内エージェントが取引先から送付されたPDF内の不可視テキストを読み込み、本来アクセス権のない社内ドキュメントを外部に送信したインシデントが業界内で共有され、大きな衝撃を与えた。

ツール連鎖攻撃（Tool Chaining Attack）――2026年の新たな主役

エージェントが複数のツールを組み合わせるアーキテクチャは、攻撃者にとって新しい攻撃面（アタックサーフェス）を提供する。**ツール連鎖攻撃**とは、個々のツール呼び出しは正当な操作に見えるが、それらが特定の順序で組み合わされることで意図しない結果を生む攻撃パターンだ。

たとえば、「①社内検索ツールでユーザーの個人情報を取得 → ②要約ツールでデータを圧縮 → ③メール送信ツールで外部アドレスに転送」という3ステップは、個別には業務上ありえる操作だが、連鎖すると情報漏洩が完成する。2026年3月にNIST が公開したドラフト文書「AI Agent Security Framework（NIST AI 800-1）」でも、この連鎖リスクに対する制御が重点項目として新設された。

さらに深刻なのが**権限昇格の連鎖**だ。ツールAの出力をツールBの認証トークンとして再利用するような設計上の欠陥が存在する場合、エージェントが意図せず（あるいは攻撃者の誘導により）権限境界を越えてしまうケースが実証されている。

防御パターン——2026年のベストプラクティス

では、これらの脅威に対して現時点でどのような防御が有効なのか。以下に実務で採用が広がっている主要パターンを挙げる。

1. ツール呼び出しのサンドボックス化と最小権限原則 エージェントに付与するツールの権限を、タスク単位で最小限に絞る。LangChainやCrewAIなどの主要フレームワークは2026年版でツールごとのスコープ制御機能を標準搭載した。「読み取りのみ」「特定ディレクトリ限定」といった粒度でポリシーを宣言的に定義できる。

2. ツール連鎖のポリシーエンジン（Chain-of-Tool Policy） 個別のツール呼び出しではなく、呼び出しの「シーケンス」に対してルールを適用するアプローチだ。たとえば「個人情報を含む検索結果が返された同一セッション内では、外部送信ツールの呼び出しを禁止する」といったポリシーを定義する。Microsoft Semantic Kernelが2026年1月にリリースしたGuardrails v2がこの分野で先行しており、DAG（有向非巡回グラフ）ベースのポリシー記述言語を提供している。

3. 入出力フィルタリングの多層化 間接プロンプトインジェクション対策として、外部データをエージェントに渡す前段に専用の検査レイヤーを置く。Anthropicが公開した「Constitutional Filtering」やGoogleの「AgentShield」は、入力データ内の命令的テキストを検出・中和する専用モデルをゲートウェイとして配置するアーキテクチャだ。

4. Human-in-the-Loopの戦略的配置 すべてのアクションに人間の承認を求めるのは非現実的だが、**破壊的・不可逆なアクション**（データ削除、外部送信、決済実行など）に限定してヒューマンゲートを設けるのは極めて有効だ。リスクス

コアリングによって動的に承認要否を切り替える仕組みが、エンタープライズ向けエージェントプラットフォームの標準機能になりつつある。

「信頼境界」の再定義が急務

従来のセキュリティ設計では、信頼境界は「システム内部 vs 外部」という比較的明確な線引きだった。しかしエージェントアーキテクチャでは、LLM自身が境界上の存在となる。外部データを解釈し、内部ツールを操作する「仲介者」であるLLMは、信頼できるコンポーネントなのか、それとも潜在的に侵害されうるコンポーネントなのか。答えは「両方」であり、だからこそゼロトラストの思想をエージェント設計に本格的に適用する必要がある。

2026年はAIエージェントが「便利なツール」から「企業インフラの一部」へと昇格した年として記憶されるだろう。そしてそれは同時に、セキュリティの責任範囲が根本から書き換わる年でもある。エージェントを本番で動かすすべてのチームが、今すぐ脅威モデリングを始めるべきだ。攻撃者は、すでに始めている。

EU AI Act本格施行から半年——日本企業が今やるべきコンプライアンス対応チェックリスト

2026年2月、EU AI Actのハイリスク分類規制がついに完全適用フェーズに突入した。そしてわずか2か月後の今、すでに最初の執行アクションが動き始めている。「うちはEU域内に拠点がないから関係ない」——その認識は、もう通用しない。EU市場にAI搭載製品・サービスを提供するすべての企業が規制対象だ。日本企業にとって「対岸の火事」ではなくなった今、最低限やるべきことを整理しておこう。

直近の執行動向——何が起きているか

2026年3月、欧州AI局（AI Office）は大手テック企業2社に対し、ハイリスクAIシステムの適合性評価文書の不備を理由に正式な情報提供命令を発出した。また、加盟国の市場監視当局がHR領域の採用AIツールに対して初の立入調査を実施したとの報道も続いている。注目すべきは、プロバイダー（開発者）だけでなく、**デプロイヤー（利用企業）** にも義務が課されている点だ。EU域内の顧客にSaaSを提供する日本企業は、デプロイヤー規制を経由して間接的に対応を求められるケースが急増している。

今すぐ確認すべき5つのチェック項目

以下は、日本企業が最優先で押さえるべき対応項目だ。

1. **リスク分類の棚卸し** — 自社が提供・利用するAIシステムをAnnex IIIのハイリスクカテゴリに照合し、該当有無を判定する。特に採用・信用スコアリング・生体認証関連は要注意。
2. **適合性評価と技術文書の整備** — ハイリスク該当のシステムについて、Art.11に基づく技術文書（学習データの概要、性能指標、バイアス検証結果等）を作成・維持しているか確認する。
3. **EU域内の代理人（Authorised Representative）の指定** — EU域内に拠点がない企業はArt.22に基づき代理人を選任する義務がある。未指定のまま市場投入を続けるのは重大なリスクだ。
4. **人間による監視（Human Oversight）体制の構築** — Art.14が求める人間介在の仕組みを運用レベルで実装し、記録を残す。形式的な「人間が最終承認」だけでは不十分とされる傾向が強まっている。
5. **インシデント報告フローの策定** — 重大インシデント発生時、72時間以内に当局へ報告する義務（Art.62）に対応できる社内フローを構築しておく。

日本国内の動きも見逃すな

内閣府のAI戦略会議は2026年春の中間取りまとめで、EU AI Actとの「相互運用性」を意識したガイドライン改定の方向性を示した。国内法制化の議論も加速しており、EU対応で整備した文書やプロセスは、そのまま国内規制への備えにもなる。

まとめ

EU AI Actの執行は「様子見」のフェーズを完全に過ぎた。まずは上記5項目を起点に、自社のギャップを洗い出すことが最初の一歩だ。規制対応はコストではなく、グローバル市場で信頼を勝ち取るための投資——そう捉え直す企業が、次の競争で勝ち残る。